

```
<!--OOP PROJECT-->
```

DistriCine:

DistriCine: Seventh Art Made Code

```
<From="Oscar Camargo,  
      Brayan Diaz,  
      Diego Yañes"/>
```

```
}
```



Contents

01

Introduction

02

Problem Statement

03

Solution

04

Objectives

05

Methodology

06

Application

07

Results

08

Conclusions

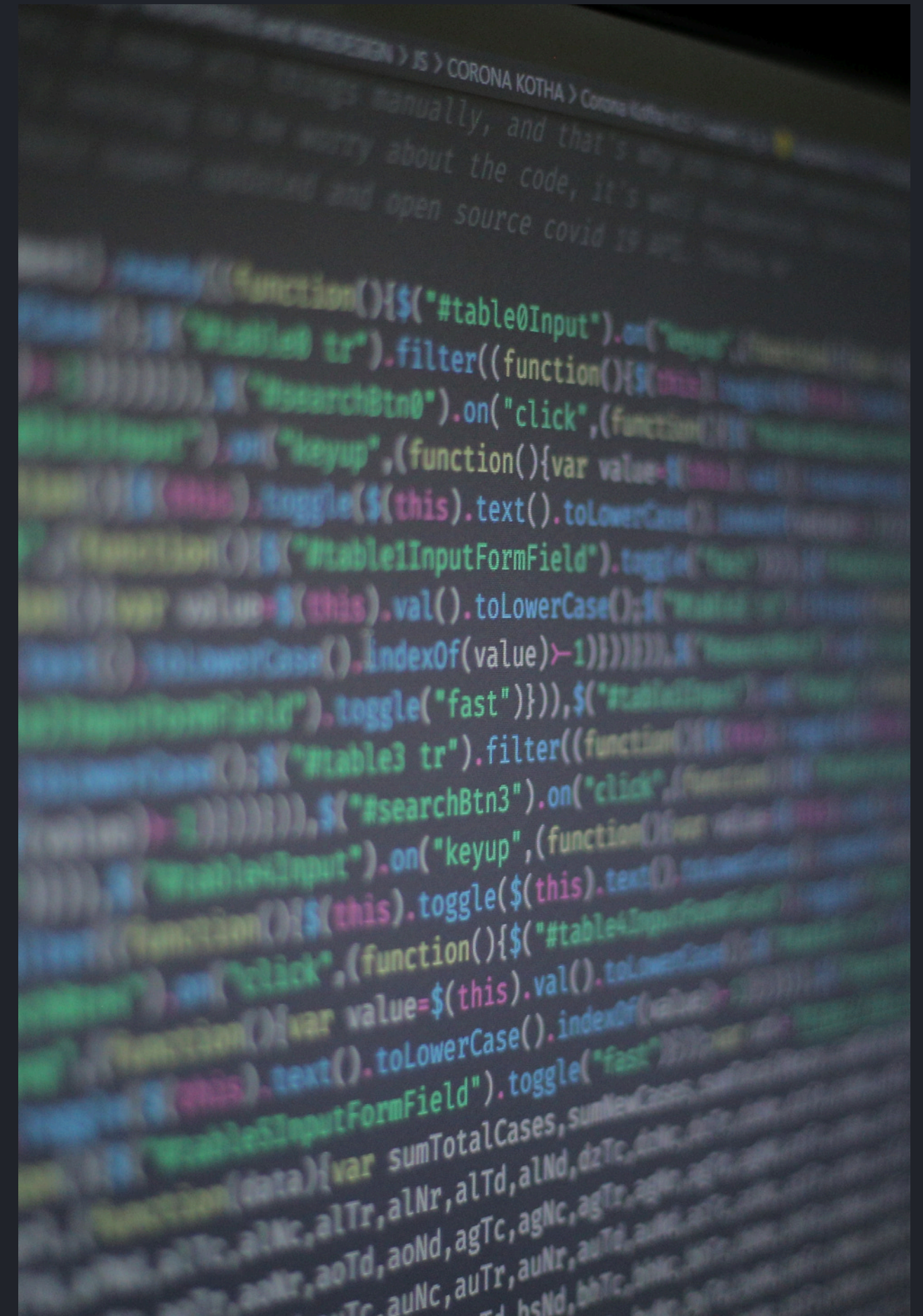
Introducción {

DistriCine digitizes the management of the university film club.

It allows users to watch films by making reservations at the university's various locations and optimizes the cultural experience at the Universidad Distrital.

An application developed using object-oriented programming principles, UML design, and other tools

}



Problem Statement {



Currently, the university manages non-academic activities using inefficient, decentralized, and poorly adapted manual methods across multiple campuses.

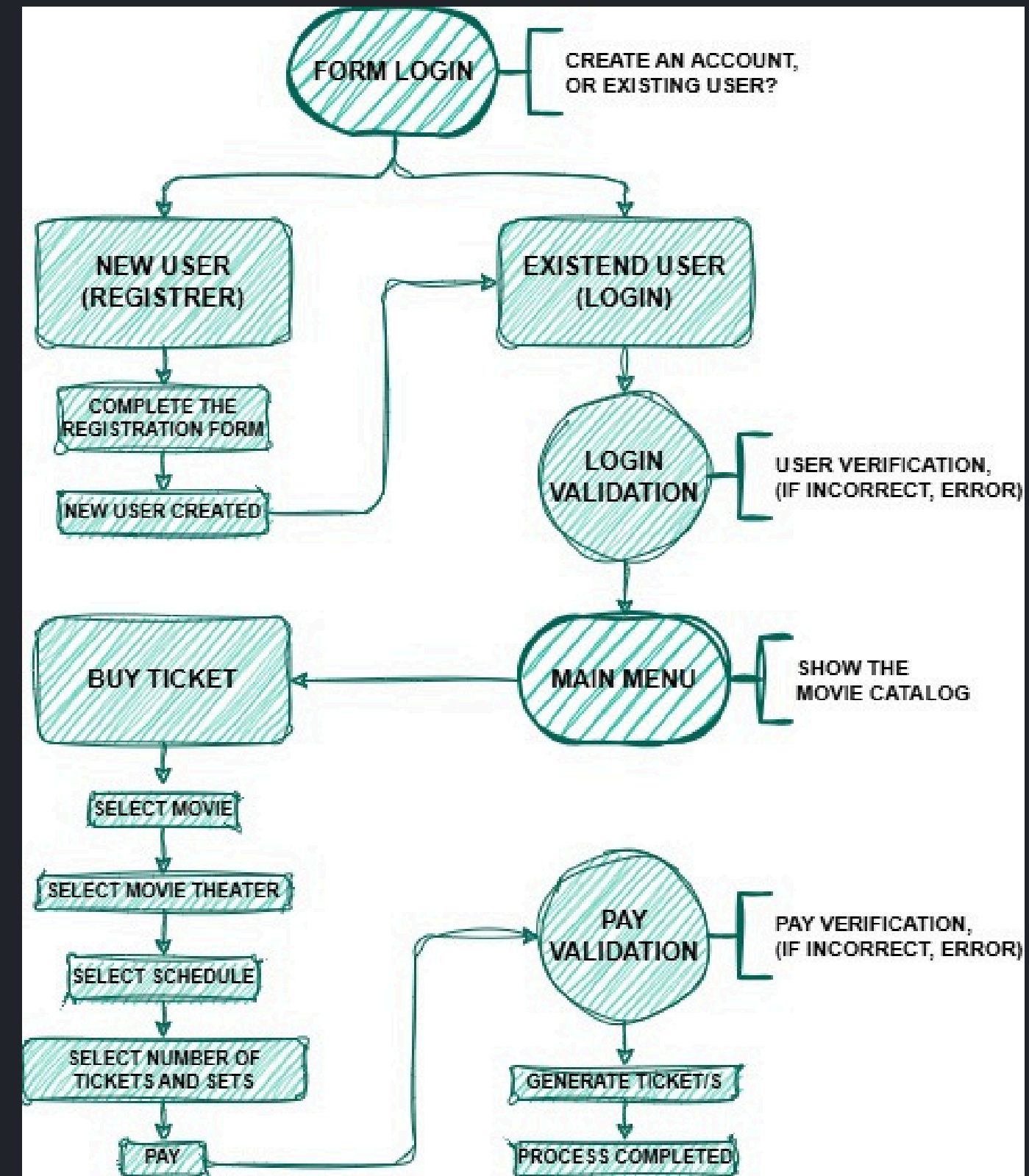
Furthermore, these activities are intended to foster socialization among students, and a sound process must be implemented to achieve this goal.

}

Solution {

The solution is a fully implemented application built with object-oriented programming. Its modular, layered architecture ensures clear responsibility separation and easy maintenance.

The UML design guided the creation of reusable classes and structured logic for managing movies, schedules, reservations, and tickets.



}

Objectives {

General:

To develop an automated application that improves the distribution and allocation of non-academic spaces at the University through a centralized film-based system.

Specific:

- To design and implement a functional, intuitive system architecture using object-oriented principles and reusable components.
- To optimize the allocation and management of film screenings across all university locations.



}

Methodology {

The project followed a structured development methodology that began with the design phase.

First, we defined the system requirements and created UML diagrams to model the architecture, relationships, and core functionalities.

Once the design was validated, we developed mockups to refine the user interface and ensure an intuitive experience. With the visual and structural model completed, we proceeded to implement the system using object-oriented programming, applying modular and layered principles. This step-by-step process ensured coherence between design and code, resulting in a functional and well-organized application.

}

Application {

During the analysis and planning phase:

- Requirements:

- 11 - Functional Requirements
- 7 - Non-Functional Requirements

- User Stories:

- 3 - Admin Stories
- 2 - Student Stories
- 1 - User Storied

- CRC CARDS:

- 10 - Cards

}

- Class Diagram:

MODULES :

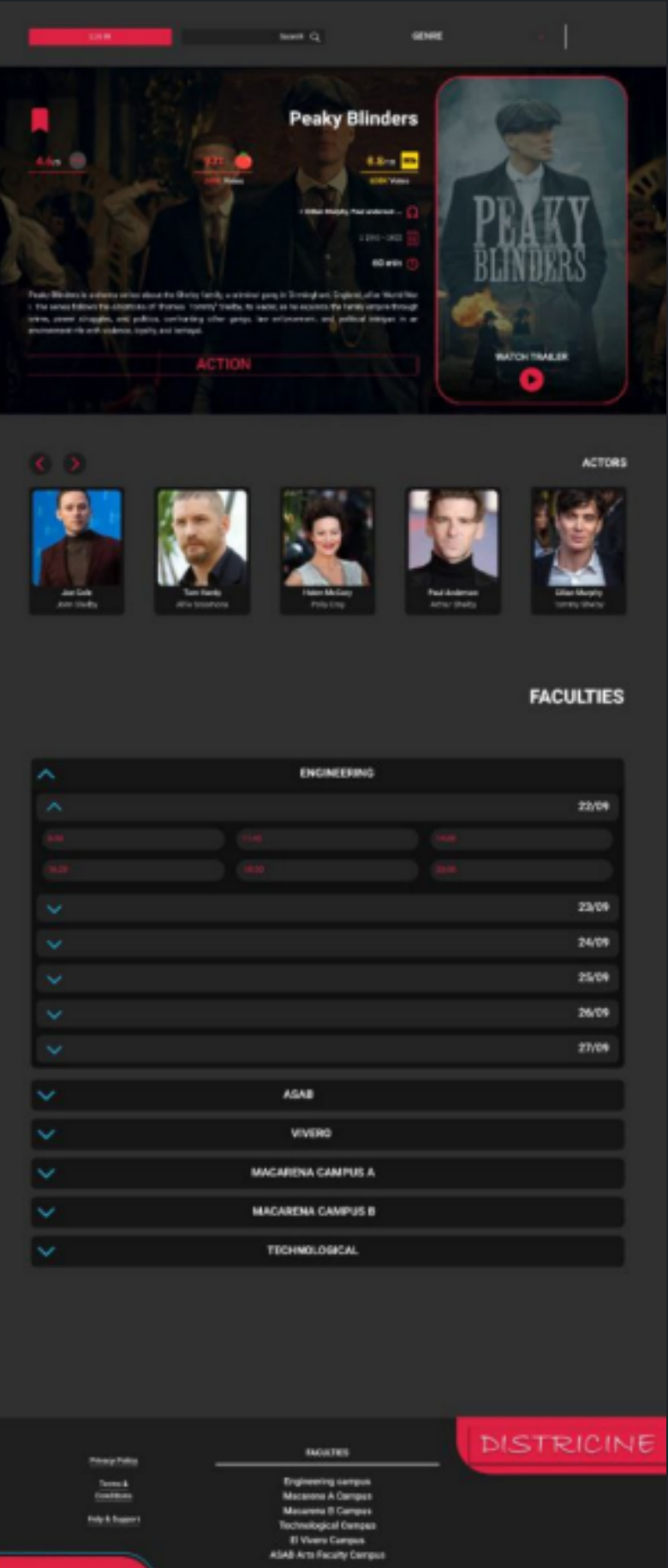
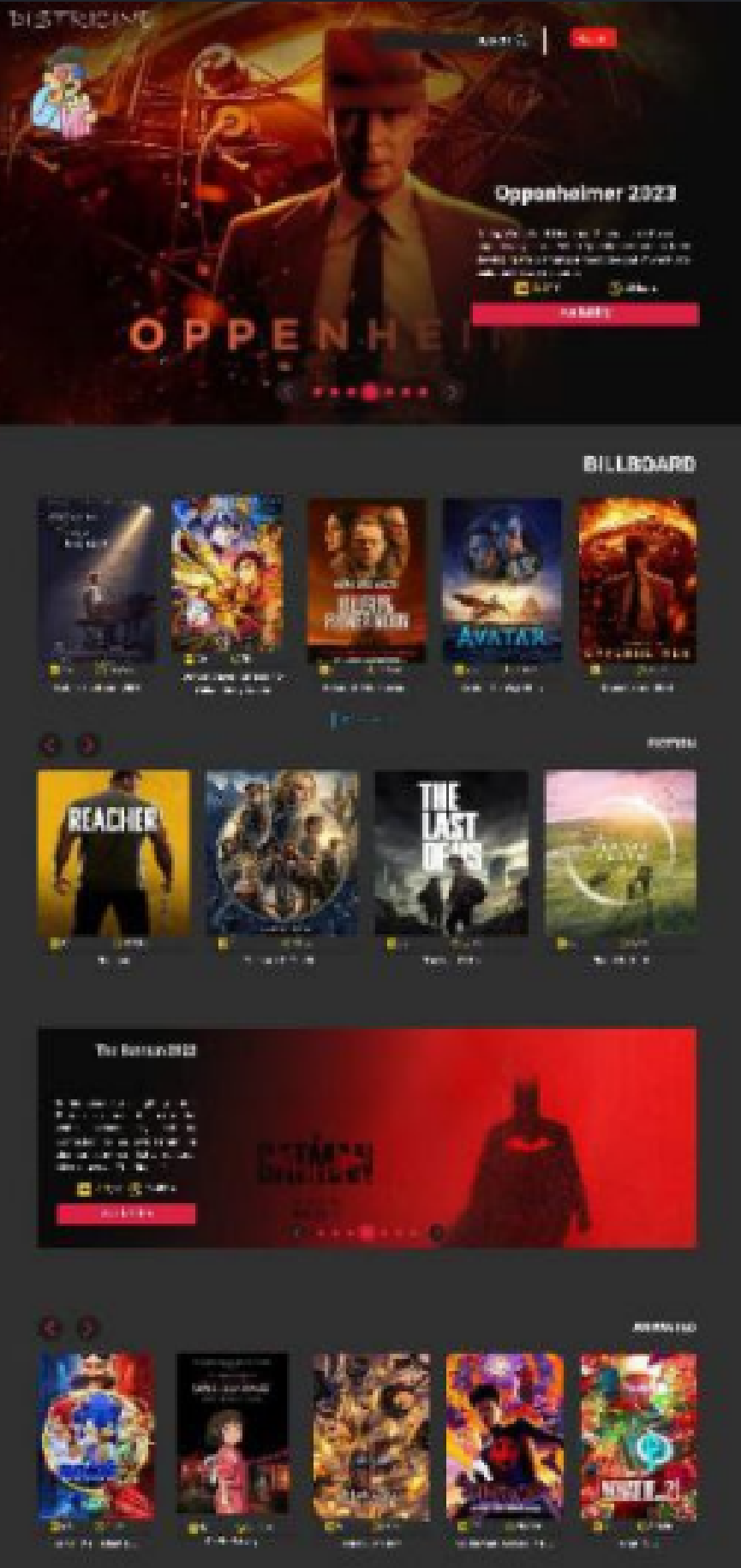
1. User (With 2 Sub-Classes)
 - Customer User
 - Admin User
2. Abstract Payment (Union of Interface and Class Payment)
3. Abstract Reservation (Union of Interface and Class Reservation)
4. Ticket
5. Movie
6. Schedule
7. Movie Theater
8. Headquater
9. Interface Show Info

}

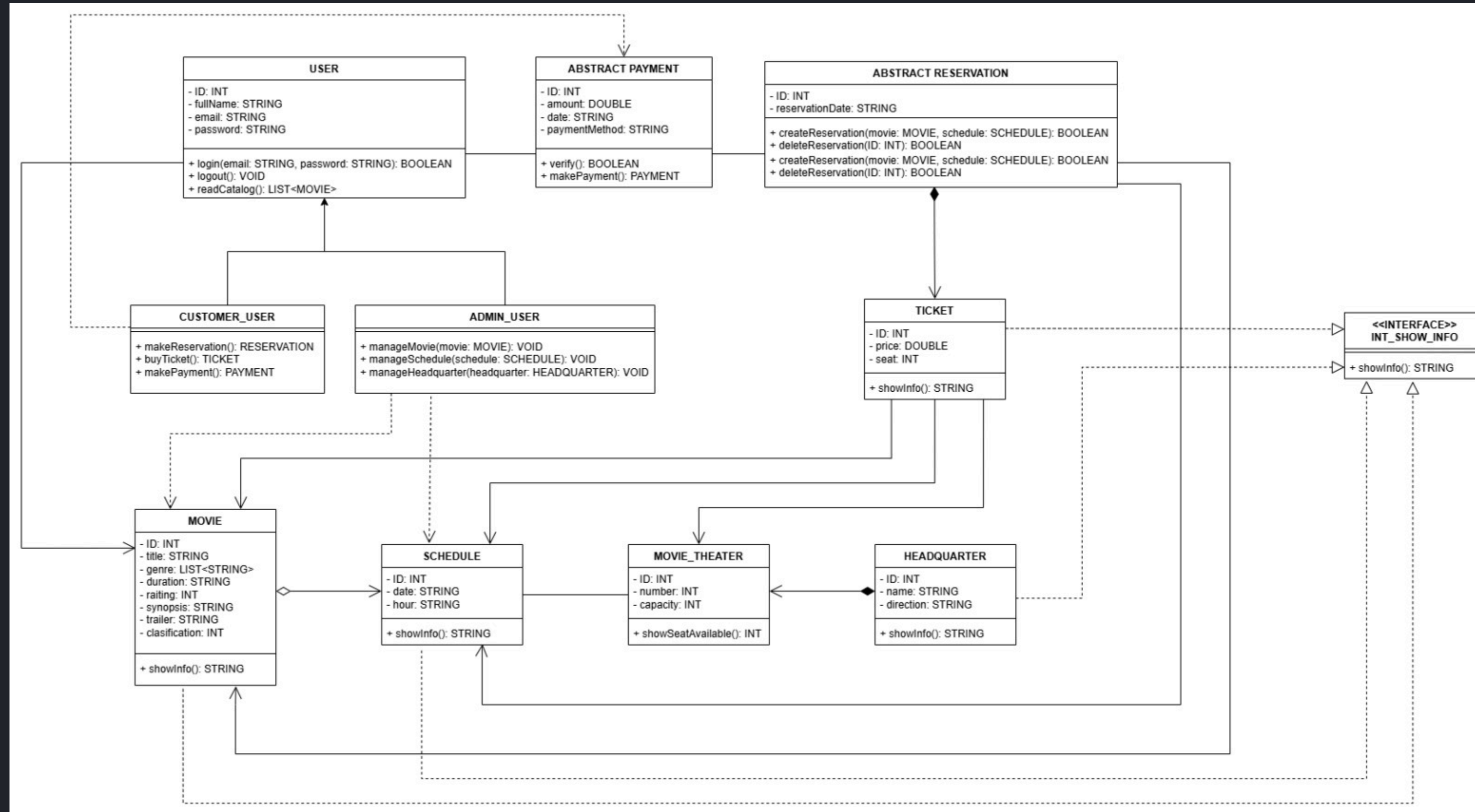
MOCK-UPS:

The mockups illustrate the visual structure and user flow of the application, allowing us to validate the interface before development.

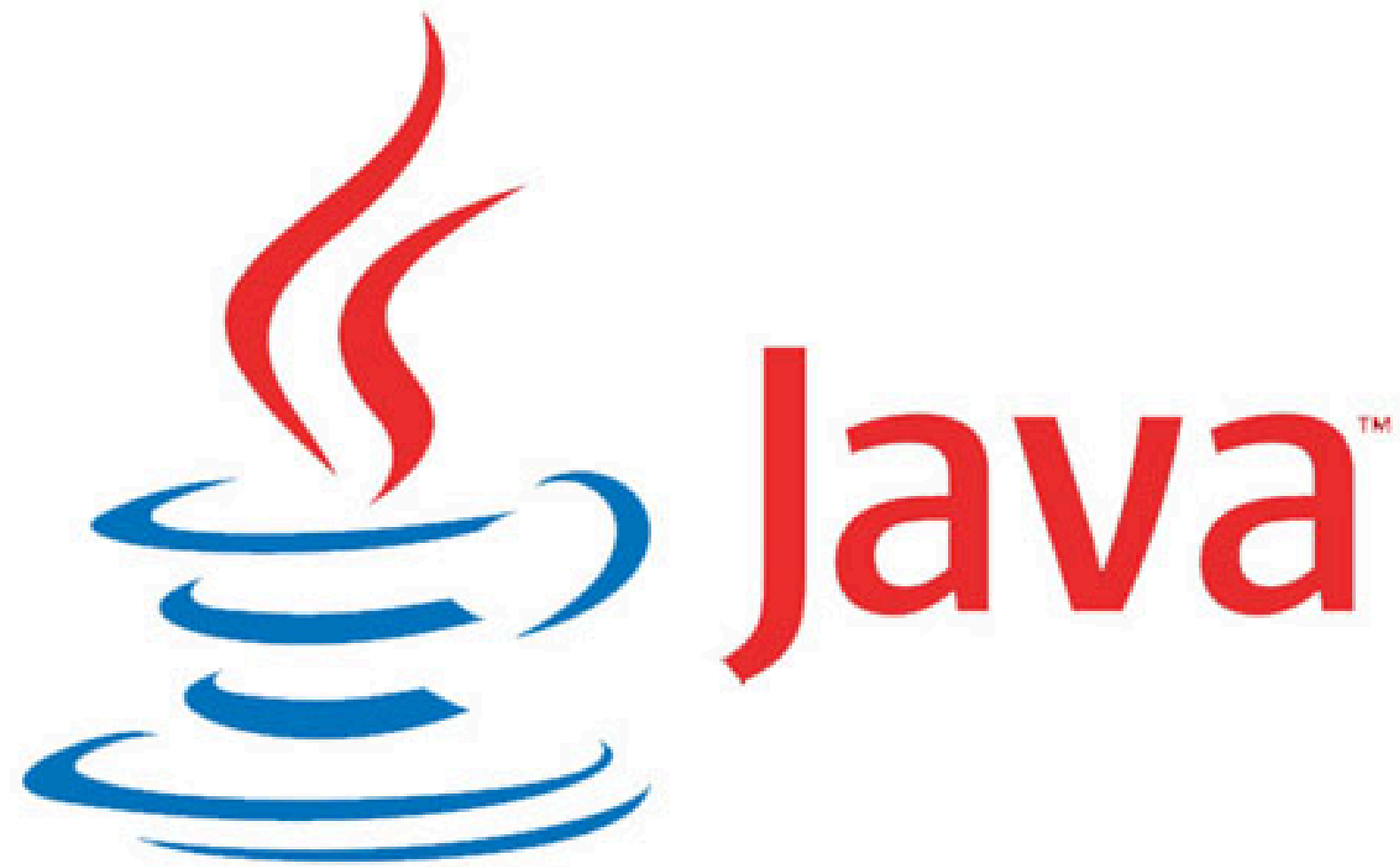
They represent the main screens of the system, showing how users interact with movies, schedules, reservations, and ticket options.



CLASS DIAGRAM:



We will show you the code in execution later.



Results {

The final application demonstrates a scalable architecture, efficient data handling, and smooth user interaction.

Usability tests confirmed an intuitive interface, while system modules for movies, schedules, reservations, and tickets operated consistently and without errors.



}

Conclusions {

- The DistriCine application provided a solution to the problem and laid the groundwork for a more robust and complex future web application.
- The console prototype validated the main functionalities, confirming the system's viability and its ability to perform the basic tasks outlined in the UML diagram, demonstrating coherence and logic.
- All phases were completed, from design (CRC, UML, Mock-ups) to OOP principles (SOLID, Layers, GUI, JavaFX) and Testing & Validation.



}

```
<!--OOP PROJECT-->
```

THANKS! {

```
<Por="Oscar Camargo,  
      Brayan Diaz,  
      Diego Yañes"/>
```

}